



**Faculdade de Design,
Tecnologia e Comunicação**
Universidade Europeia

Relatório

Implementação de comandos UNIX em Linguagem C

Curso: Engenharia Informática

Disciplina: Sistemas Operativos

Ano letivo: 2025-2026

Docente: Duarte Cavaleiro

Membros do Grupo:

José Morais (20230315)

Evandra Gomes (20230416)

Maio 2026

Índice

<i>Introdução</i>	<i>3</i>
<i>Comandos implementados.....</i>	<i>4</i>
<i>Estrutura do projeto</i>	<i>5</i>
<i>Divisão de trabalho.....</i>	<i>6</i>
<i>Tabela de systems calls</i>	<i>7</i>
<i>Dificuldades Encontradas.....</i>	<i>8</i>
<i>Conclusão.....</i>	<i>9</i>
<i>Anexos</i>	<i>10</i>

Introdução

O projeto consiste na implementação, em Linguagem C sobre Linux, de 9 comandos ao estilo UNIX. Cumprem-se os 3 comandos obrigatórios (sort, ls, UEsh/islash) e 6 dos 8 opcionais (tail, head, cp, mv, rm, kill, grep, replace).

Os comandos podem ser compilados individualmente ou em conjunto através do Makefile. Toda a funcionalidade comum aos vários comandos foi extraída para uma biblioteca partilhada (utilitarios_comuns), conforme exigido no briefing.

Comandos implementados

#	Comando	Pontuação	Descrição breve
1	<code>head</code>	2	Lista as primeiras linhas de um ficheiro
2	<code>tail</code>	2	Lista as últimas linhas de um ficheiro
3	<code>rm</code>	1	Apaga ficheiros (com verificação prévia)
4	<code>mv</code>	1	Move ou renomeia um ficheiro
5	<code>kill</code>	1	Termina um processo activo via SIGTERM
6	<code>grep</code>	2	Procura strings em ficheiros
7	<code>sort</code>	2	Ordena ficheiros de texto (output em <code>.sort</code>)
8	<code>ls</code>	3	Lista o conteúdo de um diretório
9	<code>islash</code>	3	Shell interpretador de comandos (UEsh)
Total		17 valores	

Estrutura do projeto

```
Projecto_SO/
├── Makefile
├── README.md
├── RELATORIO.md
├── include/
│   └── utilitarios_comuns.h      # API da biblioteca partilhada
├── src/
│   ├── utilitarios_comuns.c      # Implementação da biblioteca
│   ├── head.c
│   ├── tail.c
│   ├── rm.c
│   ├── mv.c
│   ├── kill.c
│   ├── grep.c
│   ├── sort.c
│   ├── ls.c
│   └── islash.c
├── bin/                          # Executáveis (gerado pelo make)
├── docs/
│   ├── manuais/                  # Manuais de cada comando (.txt)
│   └── Relatorio.pdf/            # Relatório do projeto (.pdf)
└── tests/                        # Scripts de teste (.sh)
```

Divisão de trabalho

Cada um dos membros do grupo ficou responsável por um conjunto de comandos. Tais responsabilidades incluem implementação do código e realização de testes de cada comando.

José Moraes: ls, rm, sort, islash

Evandra Gomes: tail, head, grep, kill, mv, utilitário comum.

Tabela de systems calls

Syscall	Onde é usada	O que faz
fork()	islash	Cria processo filho
execvp()	islash	Substitui processo por outro programa
waitpid()	islash	Espera que filho termine
kill()	kill	Envia sinal a processo
signal()	islash	Configura handler
chdir()	islash	Muda directório
getcwd()	islash, ls	Lê directório actual
gethostname()	islash	Lê nome da máquina
opendir/readdir	ls	Itera directório
lstat()	ls	Lê metadados de ficheiro
rename()	mv	Renomeia/move ficheiro
unlink()	rm, mv	Apaga ficheiro
access()	rm, mv	Verifica existência

Dificuldades Encontradas

- Inicialmente o **head** foi implementado com **-n NUM** (como o head real), mas relendo o briefing percebemos que **-NUM** é a sintaxe correta para contagem.
- O **tail** exigiu repensar a numeração de linhas. Tem de refletir a posição real, não a ordem de impressão.
- No **islash**, o **Ctrl+C** matava o shell até instalarmos um **handler SIG_IGN**.

Conclusão

A realização deste projeto permitiu consolidar, de forma prática, os conceitos fundamentais de Sistemas Operativos e desenvolvimento em Linguagem C no ecossistema Linux. A implementação da shell UESh e dos utilitários ao estilo UNIX proporcionou um entendimento profundo sobre a gestão de processos, system calls (como o `fork()`) e a manipulação concorrente de ficheiros.

Adicionalmente, o cumprimento dos requisitos estruturais do briefing, como a criação de bibliotecas partilhadas e a uniformização do argumento de ajuda `-h`, reforçou a importância da modularidade, legibilidade e reutilização de código num projeto de Engenharia Informática. Apesar dos desafios técnicos superados ao longo do desenvolvimento, o trabalho em equipa foi determinante para alcançar uma solução robusta, funcional e livre de erros. Em suma, o projeto superou com sucesso os objetivos de aprendizagem propostos, preparando-nos para desafios mais complexos no futuro.

Anexos

Repositório GitHub: [link](#)